High Throughput Large Integer Multiplier Generation From Optimized FPGA DSP Block Bases

Abstract

Designing high-throughput large integer multipliers requires navigating an architecture-specific design space: DSP blocks have irregular operand widths, limited cascade/shift behaviors, and multi-stage pipeline options, while large designs must balance throughput against latency and DSP usage against LUT/FF overhead. This paper presents an end-to-end automated, architecture-aware generation flow that takes a compact specification and produces validated RTL, testbenches, and implementation scripts. The flow automatically constructs DSP-efficient base multipliers, tiles them to arbitrary target sizes using Schoolbook and Karatsuba decomposition hierarchies, and inserts fast pipelined combinational logic to achieve high performance. Crucially, the tool exposes tuning parameters that let designers explore tradeoffs between throughput and latency, and between DSP usage and LUT/FF overhead, without rewriting RTL. We demonstrate full automation of large multiplier generation, achieving up to 56% higher frequency while using 17% fewer DSP blocks than prior FPGA approaches.

Keywords

Arithmetic, Multipliers, DSP blocks

ACM Reference Format:

. 2026. High Throughput Large Integer Multiplier Generation From Optimized FPGA DSP Block Bases. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (ICCAD'26).* ACM, New York, NY, USA, 9 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

While multiplication is conceptually simple, producing a fast and resource-efficient large integer multiplier on an FPGA is a design automation problem. Cryptographic applications motivate the operand sizes (e.g., 160–512 bits for ECC and 1024–4096 bits for RSA [21]), but achieving high frequency at scale requires systematic choices about base multiplier dimensions, decomposition hierarchy (Schoolbook vs. Karatsuba), DSP block cascade ability, and pipelining of both DSP block datapaths and wide shift/add logic. Manually exploring these choices is slow and error-prone, and existing generator flows often underutilize DSP architectural features or overlook the key tradeoffs between throughput and latency, and between LUT usage and DSP usage.

Fig. 1 summarizes the automated flow presented in this paper. From an input specification, it generates base multipliers mapped to architecture-specific DSP blocks, tiles them to the target size with hierarchical decomposition, schedules and synchronizes datapath pipelines, validates functionality in software, and produces synthesizable RTL and implementation scripts. This automation allows much more systematic exploration of design tradeoffs. Resulting designs can be configured either for maximum frequency (and throughput) at the cost of higher end-to-end latency in cycles, or for lower latency at the cost of reduced throughput. Second, mapping and tiling options (e.g., maximizing the number of cascaded DSP blocks vs. LUT-based shifting/accumulation) can trade off DSP count against LUT/FF overhead.

Field-Programmable Gate Arrays (FPGAs) offer a flexible architecture that can be used to implement a variety of applications in domains such as image processing [19, 36, 38], digital signal processing [6, 13, 24], and cryptography [3, 4, 12, 20, 37]. Because multiplication is a common operation and expensive to implement in LUTs, FPGAs have provided dedicated DSP blocks that deliver higher throughput and better area/energy efficiency than LUT implementations. DSP blocks also support common arithmetic patterns (e.g. multiply-accumulate). However, achieving an optimal balance of DSP blocks and LUTs for large, structured arithmetic remains challenging.

Large multipliers are typically constructed by decomposing operands into smaller tiles that match the underlying DSP block wordlengths. Schoolbook decomposition computes multiple partial products and combines them with shifts and additions, while Karatsuba decomposition [14] reduces the number of multiplication operations in exchange for wider additions/subtractions (Section 2). For extremely large sizes, FFT [5, 8, 35] and NTT [9, 15] based methods are also possible.

Effective large multiplication decomposition on FPGAs must consider the underlying architecture to achieve high performance and resource efficiency: DSP primitives are asymmetric with non-power-of-2 wordlengths, signed/unsigned handling affects tiling, and achieving high frequency requires respecting DSP pipeline constraints. FPGAs also expose additional architectural structures such as cascade paths that can improve both performance and resource usage when exploited. This paper presents an automated, architecture-aware generation flow that produces efficient base multipliers and scales them to large target sizes while enabling throughput–latency and LUT/FF–DSP tradeoffs.

The main contributions of this paper are:

(1) An end-to-end automated, architecture-aware toolchain (including an intermediate representation, software-level functional validation, and implementation reporting) that turns a compact specification into synthesizable RTL, testbenches, and scripts, and provides options to explore throughput–latency and LUT/FF–DSP tradeoffs. The open-source tool will be released with the paper.
(2) A method for building architecture-aware base multipliers that map efficiently to FPGA DSP blocks while achieving

ICCAD'26, Woodstock, NY

ACM ISBN 978-1-4503-XXXX-X/2026/06
https://doi.org/XXXXXXX.XXXXXXX

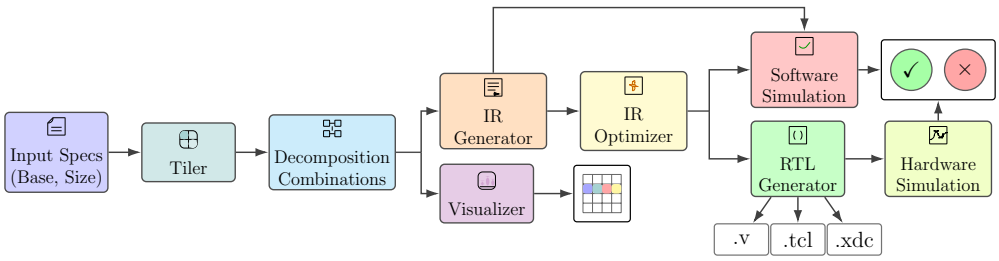


Figure 1: Overview of the automated generation flow from specification to RTL implementation and reporting.

high frequency and low resource usage, shown to outperform the vendor's 64-bit IP multiplier.

(3) Architecture-aware construction techniques for scaling base multipliers to arbitrary target sizes, including hierarchical tiling, Schoolbook/Karatsuba hierarchy selection, cascade-compatible DSP grouping, and fast pipelined reduction of wide shift/add logic.

(4) Evaluation across operand sizes and base sizes, including a 1024-bit design compared against optimized HLS implementations and state-of-the-art approaches in [25] and [2].

2 Background

2.1 Decomposition Schemes

Large multipliers are typically built by decomposing the computation across smaller multipliers with shift-and-add operations combining these to form the final result.

Schoolbook Decomposition forms a $2n \times 2n$ product using four $n \times n$ partial products (Equation 1). Let $A = (A_1 \ll n) + A_0$ and $B = (B_1 \ll n) + B_0$; with reference to Fig. 2a:

$$\begin{aligned} Schoolbook(A \times B) = & P1 + (P2 \ll 2n) \\ & + ((P3 + P4) \ll n) \end{aligned} \tag{1}$$

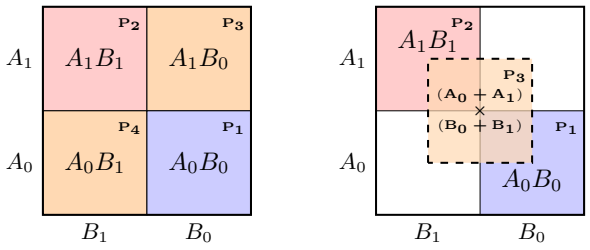
This approach scales to more splits (e.g., nine multipliers for a 3×3 split) and relies on more shift-and-add logic to combine partial products. Equation 1 can be rearranged to trade an addition for zero cost concatenation [25]:

$$(P1 + (P2 \ll 2n)) = \{P2, P1\} \tag{2}$$

Karatsuba Decomposition [14] reduces the number of multiplications needed in the 2×2 case from four to three by replacing one multiplication with additions/subtractions. One product is typically $(n{+}1)$ bits wide. Fig. 2 compares Schoolbook and Karatsuba for a $2n \times 2n$ multiplier. Karatsuba can be extended to larger grids (e.g., 3×3), saving three multipliers relative to Schoolbook (which uses 9 multipliers).

$$Karatsuba(A_0B_1 + A_1B_0) = P3 - P1 - P2 \tag{3}$$

The $(n{+}1)$-bit multiply can be implemented using an n-bit multiplier plus additional logic by splitting each operand into an n-bit part and a 1-bit part (Fig. 3) to maximally use a fixed-size component multiplier for all products. The $n \times 1$ and 1×1 terms reduce to AND gates, and non-overlapping terms can be concatenated



(a) Schoolbook decomposition. **(b) Karatsuba decomposition.**

Figure 2: Comparison of Schoolbook and Karatsuba decomposition schemes for a $2n \times 2n$ multiplier.

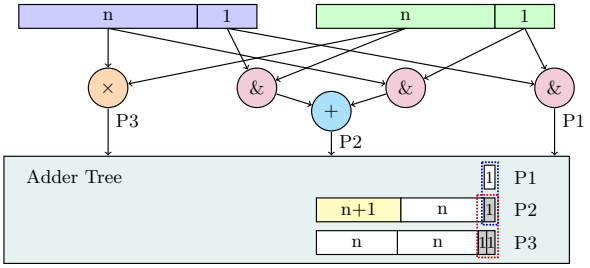
(Equation 2). Placing the $n \times n$ multiplier on the LSB side (Fig. 3b) reduces the widest adder from $2n$ to $n{+}1$ bits.

2.2 DSP Block Considerations

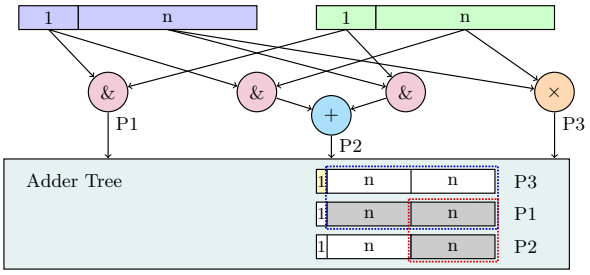
Overview and Applications. A DSP block is a hard FPGA primitive optimized for high-frequency multiply-add/accumulate operations. Modern architectures have increased multiplier widths (e.g., 25×18 in Virtex-5/6/7, 27×18 in UltraScale(+), and 27×24 in Versal) [28, 30, 33, 34]. They are widely used in FIR filters, ML inference, and scientific signal processing [10, 11, 26, 27]; some generations also support floating-point modes [18].

AMD DSP Block Architecture. An AMD DSP48E2 block integrates a pre-adder (not shown nor used in this work), multiplier, multiplexers, and an ALU (Fig. 4). Control signals (e.g., OPMODE/ALUMODE) select the datapath/ALU expression [32], and optional pipeline registers trade latency for throughput (enabling all four registers yields maximum throughput). Cascaded ports (e.g., PCIN/PCOUT) support efficient column-wise chaining. Correct multiplier and cascade behavior requires routing the multiplier partial products to the X and Y multiplexers while selecting the appropriate Z source (e.g., PCIN) via OPMODE.

Chaining DSP Blocks. More than one DSP block can be chained to create wider multipliers. For instance, two DSP blocks can be chained to perform a 44×18 multiplication. Assuming two inputs K of size 44 bits and M of size 18 bits, K is split into two parts: the lower 17 bits (K_L) and the upper 27 bits (K_H). To fit within the 48-bit datapath and leverage the built-in 17-bit right shift, the accumulation is rewritten as follows:



(a) The n-bit multiplier is used for the MSB.



(b) The n-bit multiplier is used for the LSB.

Figure 3: $n+1$-bit multiplier used in Karatsuba decomposition. The grey areas denote the shift amount (added zeros). The dotted rectangles denote concatenation. The LSB variant reduces the widest adder from $2n$ to n+1 bits.

$$P1 = (K_L \times M), \quad P2 = (K_H \times M)$$
$$K \times M = \{(P2 + (P1 \gg 17)),\ P1[16:0]\} \tag{4}$$

Fig. 4 contrasts two chaining options: the PCOUT-to-PCIN cascade path versus routing P-to-C. With PCOUT-to-PCIN, S1 cascades PCOUT to S2 PCIN and uses the built-in 17-bit right shift in S2. Signal synchronization requires delaying K_H, M, and the extracted LSBs from P1, resulting in a 5-cycle latency, but no additional hardware usage. Because the cascade path supports only 0 or 17-bit shifts, other shift amounts must use the P-to-C path which requires shift/sign-extension (ExSRn) and routing S1's P to S2's C, typically adding an extra pipeline stage for signal synchronization.

2.3 Signed vs. Unsigned Multiplication

When constructing large multipliers from smaller ones, signedness must be handled carefully. In decomposition, only the multipliers dealing with MSBs should be signed (for a signed multiplication); all other partial products must be computed unsigned even if the overall multiplication is signed. A signed n-bit multiplier can be used as an unsigned $(n-1)$-bit multiplier by zero-padding the MSB.

2.4 IP Multipliers

AMD provides an IP multiplier in Vivado [29] that generates parameterized signed/unsigned multipliers up to 64×64 bits, with options to optimize for frequency or resource utilization and a recommended pipeline depth. For sizes larger than 47×47 bits, only the performance-optimal design is offered; it achieves maximum DSP frequency but consumes more DSP blocks than necessary.

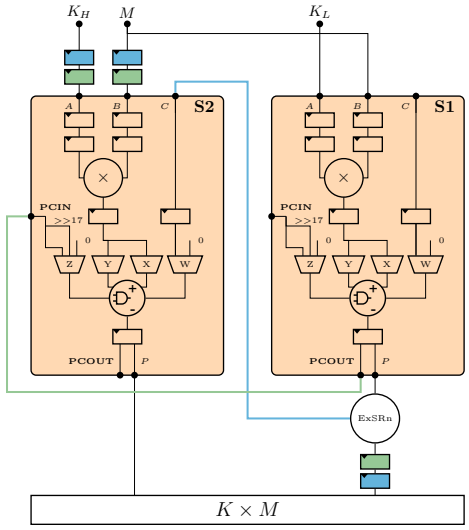


Figure 4: Comparison of DSP48E2 cascade paths: PCOUT-to-PCIN (green) vs. P-to-C (blue). The ExSRn block extracts n bits from P to output while shifting P right by n bits, sign-extending, and routing the result to the C input. The P-to-C path requires additional logic and an extra pipeline stage.

3 Large Multiplier Design

Our approach focuses on building efficient base multipliers using DSP blocks, then hierarchically tiling them up to large sizes while considering the FPGA architecture and DSP block features and using fast pipelined combinational logic.

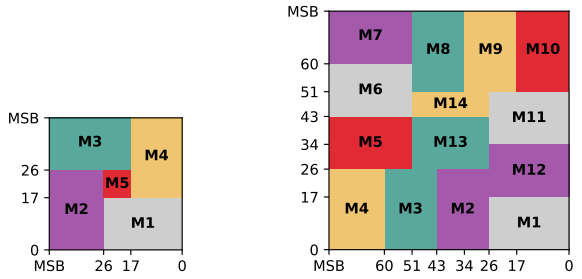
3.1 Building Base Multipliers

To construct large multipliers efficiently on FPGAs, we build base multipliers and tile them to larger sizes. Karatsuba decomposition can only combine partial products that are aligned to the same shift amount. For a $W_1 \times W_2$ base, shifts take the form $W_1 n + W_2 m$ and Karatsuba-friendly shift amounts take the form $k \cdot \frac{W_1 W_2}{\gcd(W_1, W_2)}$, where k is an integer. Thus, rectangular bases only yield sparse Karatsuba-friendly alignments (e.g., $26 \times 17 \Rightarrow 442k$). Square bases yield more Karatsuba-eligible partial products (e.g., $44 \times 44 \Rightarrow 44k$), allowing more opportunities for reducing the number of multipliers used.

A square base multiplier can be obtained using a tiling algorithm inspired by Roy et al. [23]. Given DSP primitive unsigned dimensions $W_1 \times W_2$ (e.g., 26×17 for the DSP48E2), and a target size M, the algorithm chooses integers n, m so that the covered width

$$\bar{M} = n \cdot W_1 + m \cdot W_2.$$

where $\bar{M} \geq M$. Tiles of size $W_1 \times W_2$ and $W_2 \times W_1$ are placed along the boundary of the $\bar{M} \times \bar{M}$ square, leaving a smaller square region



(a) 44-bit tiling (5 DSP48E2). (b) 78-bit tiling (14 DSP48E2).

Figure 5: Tiling schemes for different square base multiplier sizes.

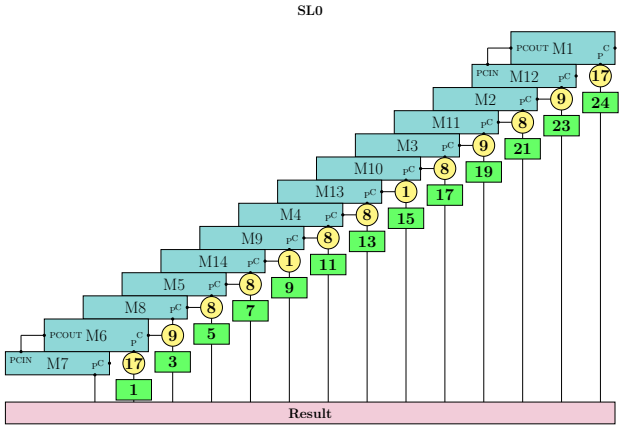
in the middle that is recursively tiled in the same way. However, the original algorithm has practical limitations that our work addresses:

- The remaining middle region is not guaranteed to be a square for arbitrary n, m.
- Hardware mapping is not architecture-aware (e.g., cascade/shift constraints), and does not explicitly provide variants that trade off LUT usage and latency.

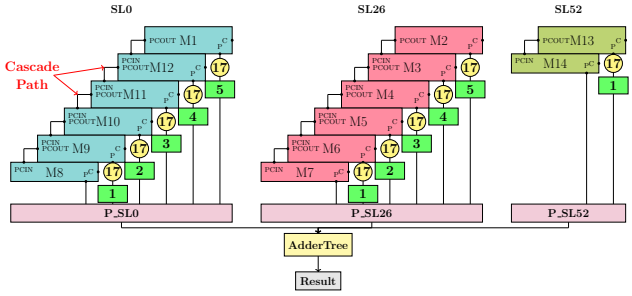
To guarantee a square middle region, we constrain one tiling coefficient to 1 (either $n = 1$ or $m = 1$). Fig. 5 shows the tiling scheme for 44- and 78-bit signed base multipliers. To address the remaining issues, we separate generating multiplication terms (tiles) from mapping them to DSP blocks. This enables architecture-aware mapping (cascade/shift constraints) and optimization for either LUT minimization or latency. Additionally, small multipliers are automatically mapped to LUTs to save DSP resources for larger widths, and all signals are automatically synchronized. Fig. 6 compares two mapping strategies for the 78-bit multiplier. First, the one-group strategy (Fig. 6a) chains as many terms as possible (sorted in ascending order based on their shift amount) to minimize LUT usage. This ordering allows for non-overlapping bit extraction and keeps additions within the DSP48E2 48-bit datapath. The multi-group strategy (Fig. 6b) forms independent chains whose terms are shifted apart by 17 bits, maximizing usage of cascade connections and enabling parallel evaluation to reduce latency.

Table 1 compares the resource usage and latency of the 44- and 78-bit signed base multipliers built using the two grouping strategies. The one-group strategy generally uses fewer LUTs, while the multi-group strategy reduces latency at the cost of slightly increased LUT usage for the adder tree. In the 78-bit one-group variant, the increased LUT usage results from the synthesis tools using shift register LUTs to synchronize datapath signals instead of FF registers.

To test the effectiveness of this base multiplier design, we compare against the AMD Vivado IP 64-bit unsigned multiplier discussed in Section 2.4. We compare it against the closest size obtainable by our base tiling algorithm: a 77-bit unsigned multiplier (implemented by zero-padding the MSB of the 78-bit multiplier). Our base multiplier achieves a slightly higher frequency than the IP, supports larger inputs, and uses 2 fewer DSPs at the cost of extra 210 LUTs. To measure DSP block utilization, we set the theoretical



(a) One group strategy (minimizing LUTs).



(b) Multi-group strategy (minimizing latency).

Figure 6: Grouping strategies for the 78-bit base multiplier. Green rectangles indicate pipeline stage counts. Yellow circles show the number of LSBs extracted from P and the corresponding right shift amount passed to C when using the P-to-C path. The multi-group strategy reduces latency, while the one-group strategy minimizes LUT usage.

Table 1: Comparison of different base multiplier strategies.

Size	Primitive Strategy	LUTs	FFs	DSPs	Latency (Cycles)
44	One group	78	323	5	9
	Multi-group	105	334	5	8
78	One group	537	1051	14	28
	Multi-group	429	1386	14	14
64	AMD IP	219	722	16	18

minimum number of DSP blocks to tile an $S \times S$ multiplier to be $\frac{S^2}{26 \times 17}$. The 64×64 IP exceeds this bound by 6.7 blocks, while our 77×77 design is within 0.59 blocks of it (sacrificing only a few bits in one block). Fig. 7 compares the two base designs when tiled to larger sizes using 2×2 and 3×3 Schoolbook and Karatsuba decompositions. When tiled at one level of hierarchy, our 77-bit base multiplier consistently uses fewer DSPs and comparable LUTs, while achieving a higher frequency.

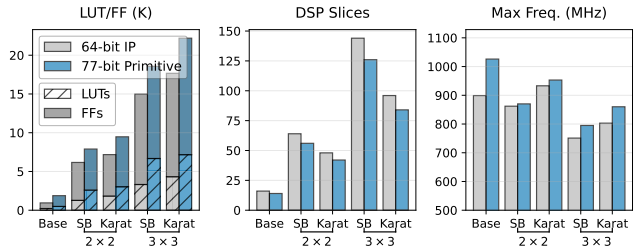


Figure 7: Resource and frequency comparison between the 64-bit AMD IP and our 77-bit base multiplier when tiled to larger sizes.

3.2 Tiler

Finding the optimal placement of small rectangles to tile a large rectangle is NP-complete [1]. General tiling schemes suffer from two main limitations when applied to large multipliers on FPGAs:

- They do not prioritize hierarchical regions that enable Karatsuba-style factoring.
- They ignore architecture-specific features such as DSP cascade capabilities and efficient base sizes.

We address these limitations by hierarchically replicating an efficient square base multiplier of size $B \times B$ (Section 3.1) using 2×2 or 3×3 grids that preserve shift alignment for systematic Karatsuba decomposition. Accordingly, the possible scaled-up side lengths are

$$H = 2^x \times 3^y \times B,$$

where each factor of 2 or 3 corresponds to one hierarchy level.

The total DSP usage can be determined once the base and decomposition schedule are chosen. Let D_{base} be the DSP usage of the base multiplier, then for a hierarchical side length $H = 2^x \times 3^y \times B$, the total DSP utilization is:

$$D_{total} = D_{base} \times \prod_{i=1}^{L} \left(\frac{s_i \times (s_i + k_i)}{2^{k_i}} \right)$$

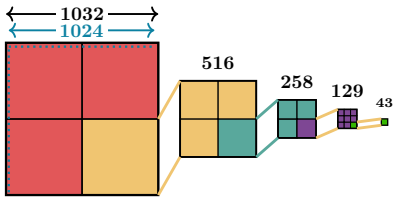
where $L = x + y$ is the total number of hierarchy levels. At each level i, $s_i \in \{2, 3\}$ for 2-way or 3-way split, and $k_i \in \{0, 1\}$ for Schoolbook or Karatsuba decomposition.

We evaluate two tiling methodologies, compared in Fig. 8 for a 1024-bit target multiplier:

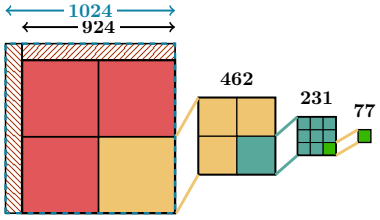
Overshoot tiling. Instead of forcing an exact target size match, we choose an achieved size $\bar{T} \geq T$ that increases the hierarchical portion (e.g., by selecting a larger H or using a base B with better $2^x \times 3^y$ coverage). This can provide more opportunities for Karatsuba decomposition and thus reduce DSP usage, at the cost of extra LUT-based shift-and-add logic to handle the excess bits.

Target-constrained tiling. Alternatively, given a target side length T, we first select the largest hierarchical side length $H \leq T$ of the form $H = 2^x \times 3^y \times B$. This produces a hierarchical $H \times H$ region that can be decomposed cleanly across hierarchy levels; the remaining non-hierarchical rectangular regions are then filled using additional placements.

Tiling the remaining area. We explore three variants for this: (1) greedily placing the same base until it no longer fits, then placing



(a) Overshoot tiling with 43-bit base showing the final achieved size exceeds the target.



(b) Target-constrained tiling with 77-bit base, showing a remaining portion that must be tiled with other multipliers.

Figure 8: Comparison of overshoot and target-constrained tiling for a 1024-bit multiplier. Overshoot tiling exceeds the target size but maximizes hierarchical structure. Target-constrained tiling stays within the target, filling the remainder with additional multipliers. Each hierarchy level can be implemented using Schoolbook or Karatsuba decomposition.

DSP-sized multiplications; (2) recursively tiling the remainder using other efficient bases (e.g., 44×44 and 94×94) and small DSP-sized tiles; and (3) a cascade-compatible variant that prefers placements enabling DSP cascade connections. In all variants, each candidate placement is scored by DSP count and by estimated LUT/FF overhead for accumulation. The cascade-compatible variant is preferred in Fig. 8b because it reduces LUT-based shifting and addition while also lowering DSP utilization (464 DSP48E2 blocks versus 507 for the greedy approach and 516 for the non-cascade-compatible variant for the 1024-bit multiplier in Fig. 8).

3.3 Optimal Base Size Choice

Base multiplier size strongly affects DSP usage, hierarchy depth, and the amount of irregular remainder tiling (Section 3.2). We evaluate the candidate bases produced in Section 3.1. Bases derived by fixing the larger DSP dimension (26 for DSP48E2 26×17) consistently reduce DSP waste (e.g., 43, 60, 77, 94, 111, 128, . . . , 264). Within this family, the 77-bit base exhibits the lowest waste when considered in isolation. However, low DSP waste alone does not guarantee an efficient hierarchy for Karatsuba or Schoolbook decomposition. We therefore also consider nearby sizes that improve hierarchical coverage, even if they yield a slightly larger achieved size. For example, tiling the 77-bit base towards a 1024-bit target yields a largest hierarchical block of 924-bits and leaves two non-hierarchical rectangular regions. In contrast, a 43-bit base can reach 1032-bits and thus increase the hierarchical nature for Karatsuba (Fig. 8).

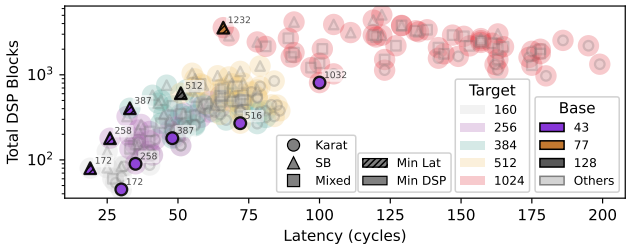


Figure 9: DSP block usage and latency tradeoffs for different base sizes across target multiplier sizes (log scale y-axis). Annotations indicate the achieved sizes. The 43-bit base with Karatsuba decomposition at all levels consistently minimizes DSP usage.

Since DSP usage and latency are known at design time, we perform a Pareto analysis across targets from 160 to 1024 bits with different bases and decomposition schemes. Fig. 9 shows that the 43-bit base with Karatsuba decomposition at all levels minimizes DSP usage at the cost of higher latency, while 77- and 128-bit bases with Schoolbook decomposition reduce latency but increase DSP consumption.

3.4 Fast Pipelined Combinational Logic

The wide combinational paths implementing logic like shift/add/-subtract can dominate the critical path in large multipliers. Therefore, we split the logic into smaller pipelined segments to achieve high throughput. Empirically, we observe a frequency drop when the segment width exceeds 64 bits. Our flow makes segment size configurable to trade off depth, latency, and timing. These intermediate logic segments are pipelined to achieve high throughput; this is done not only at the module level, but also with awareness of the pipeline schedule of individual signals, interleaving pipelining where necessary to reduce overall latency.

3.5 Automating the Flow

To enable rapid design-space exploration, we built an automated flow that integrates the components described in the previous sections. Figure 1 summarizes the end-to-end tool flow discussed in this subsection.

Input Specification. The flow starts from a compact specification that includes the target multiplication size, base multiplier dimensions and, optionally, the desired decomposition strategy (Schoolbook, Karatsuba, or mixed) and base combinational logic width. This specification is architecture-aware: it captures enough information to guide both arithmetic decomposition and downstream hardware mapping.

Tiler. The Tiler first instantiates a flat grid that covers the target multiplication using the base multiplier. When the target dimensions are not exact multiples of the base size, the remaining regions are filled using DSP block multipliers (Section 3.2), so full operand coverage is preserved.

Decomposition Combinations. A flat grid provides limited structure for higher-level decompositions. The Hierarchy Generator rewrites it into alternative hierarchical decompositions by composing 2×2 and 3×3 groups across multiple levels. For example, a 12×12 grid can be rewritten as four 6×6 blocks, where each 6×6 block is itself decomposed into four 3×3 bases (three hierarchy levels; see Fig. 8b).

Intermediate Representation (IR) Generator. Each decomposition variant is processed by the intermediate representation (IR) generator and Visualizer in parallel. The generator produces an *untimed* IR that captures arithmetic functionality and hierarchy before scheduling. Modules have explicit input/output interfaces; operations are represented as arithmetic nodes (e.g., add/sub/mul/shift/slice/concatenate); and each signal carries explicit wordlength and signedness metadata. Also, each module includes a generated testbench block that captures the expected behavior for software simulation and is used later for RTL verification. The IR tags primitive multipliers that fit DSP blocks, records pa andrtial-product shift/alignment used in later reduction stages. Keeping the IR untimed enables algebraic rewrites, adder-tree balancing, and cascade-aware grouping before clock-cycle decisions are made, all of which the tool exploits.

Software Simulation. In the software simulation stage, the tool processes the untimed IR in pure Python and compares against the testbench golden outputs. For each block, it runs randomized validation vectors (configurable count and seed) and reports detailed mismatches (inputs, expected values, computed values). This stage catches arithmetic, masking, and decomposition errors early, reducing debug cost before RTL and implementation runs.

Visualizer. The Visualizer renders each decomposition as a layout/graph view and checks geometric correctness, including 100% coverage and zero overlap constraints. This provides an immediate sanity check for legality before hardware-specific transformations proceed.

IR Optimizer. Starting from simulation-validated IR, the optimizer applies a configurable multi-pass transformation pipeline that preserves arithmetic equivalence while improving synthesis efficiency. At the program level, it can reorder blocks by dependency and add optimization metadata headers for traceability. At the block level, it performs DSP-aware rewrites (e.g., converting eligible multiplications to DSP primitive multiplications), differentiates between DSP families (e.g., DSP48E2 vs. DSP58), and captures operand swapping to match asymmetric DSP ports. It identifies cascade-friendly accumulation patterns so we can leverage the DSP cascade path to reduce LUT usage and latency. It also replaces one-bit multiplications with equivalent bitwise AND gates, and recursively optimizes nested modules. It also supports *lazy accumulation* by deferring additions so partial sums can later be folded into concatenations (e.g., using Equation 2) or globally-balanced adder trees rather than greedily instantiating wide adders early. Together with target-specific legality checks (e.g., architecture-dependent cascade and shift constraints), it produces an optimized untimed IR with explicit architecture-aware annotations that guide RTL generation. Finally, software simulation is executed again on the optimized IR to validate that the transformations preserve functional correctness. After this stage, the IR is considered fully validated and ready for RTL generation.

RTL Generator. The RTL Generator lowers the optimized IR into timed, synthesizable Verilog modules and submodules with cycle-accurate semantics. At this stage, operation scheduling, pipelining,

and synchronization are introduced according to the architecture-aware rules from Section 3.1 and Section 3.4, and testbenches are produced for functional verification. Each signal carries timing information, and operation outputs are timestamped from input availability and operator latency, so pipelining and synchronization behavior are explicitly modeled. It translates arithmetic nodes into synthesizable Verilog, inserting pipeline registers and synchronization logic as needed to meet timing and preserve correctness. It translates module calls into module instantiations with timing information, and schedules operations within each module according to the timing rules and architecture-aware constraints. This is achieved by propagating timing information through the call graph, enabling modular reuse while preserving end-to-end schedule consistency. The timing information is used to generate cycle-accurate source files and corresponding testbenches for every module in the IR to validate the final RTL against expected behavior. It also produces annotated dataflow diagrams that indicate operation grouping and pipeline synchronization points. It also generates Tcl scripts to run FPGA synthesis/implementation and collect area/timing metrics.

Hardware Simulation. The generated RTL and testbenches are simulated using a hardware simulator (e.g., Vivado Simulator) to validate cycle-accurate behavior. Every module has a corresponding testbench and expected timing behavior, so failures can be traced back to the IR timing model. The resulting timelines are used to derive latency estimates, generate cycle-accurate verification expectations, and drive artifacts and resource/performance reporting.

Implementation and Reporting. The generated source files are synthesized and implemented on the target FPGA using the generated Tcl scripts. After implementation, the tool collects area and timing metrics and generates reports across configurations, including DSP/LUT/FF usage, frequency, latency, and normalized area. This enables systematic comparison across base sizes, decomposition strategies, and tiling methodologies to identify optimal design points for large multipliers on FPGAs.

4 Evaluation

4.1 Experiments and Results

We target the AMD Alveo U280 FPGA (XCU280 part). We generate large multipliers for operand sizes from 160 to 1024 bits using overshoot tiling with a 43-bit base and target-constrained tiling with a 77-bit base as discussed in Section 3.2. For the 1024-bit case, we also applied our method to the 64-bit AMD IP multiplier as a base in which overshoot and target-constrained approaches yield the same result (four levels of 2×2 grids). Wide additions are pipelined using 64-bit segments. We report frequency, LUTs, and DSPs obtained from Vivado implementation post-place-and-route. We also report a normalized area metric as used in [22, 25]. In general, given a device-specific LUT-to-DSP ratio ρ, we normalize area as

$$A_{\text{norm}} = \#\text{LUTs} + \rho \cdot \#\text{DSPs}.$$

For the U280 (1082K LUTs and 8490 DSPs [31]), we have $\rho = 126$.

Fig. 10 summarizes DSP usage and normalized-area trends across decomposition schedules for the 77-bit and 43-bit bases. Across targets, increasing the number of Karatsuba levels generally reduces DSP usage and normalized area, and often improves frequency. The

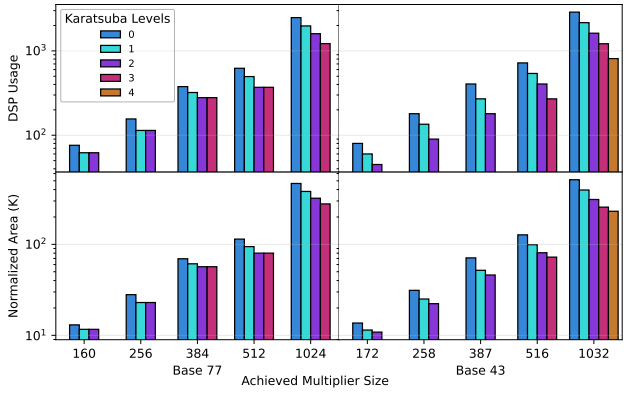


Figure 10: Resource utilization for target-constrained tiling using our 77-bit base and overshoot tiling using our 43-bit base multipliers, with varying numbers of Karatsuba levels. 77-bit base is favorable for fewer Karatsuba levels, while 43-bit base benefits from deeper Karatsuba hierarchies. Note the logarithmic y-axis.

43-bit base benefits more from deeper Karatsuba hierarchies due to its larger number of hierarchy levels. The 77-bit base is typically competitive when fewer Karatsuba levels are used.

We compare against IMpress (an HLS framework that uses E-graph-based rewriting at different hierarchy levels to select the decomposition) [25], including the HLS baseline and both IMpress configurations reported in their work, and against the beam-search tiling heuristic from [2]. Table 2 lists the lowest DSP utilization configurations generated by our tool per target size and base; other configurations are omitted for brevity. Our tool achieves significantly higher frequency (and hence throughput) than previous work, regardless of the base used. Using the 64-bit IP multiplier as a base wastes resources, while our 77- and 43-bit base multipliers dramatically reduce DSP block usage.

Fig. 11 compares the frequency of our implementations with other work against both DSP count and normalized area. The HLS design uses the fewest DSPs but has low frequency and high normalized area. IMpress uses moderate resources but also runs at low frequency. Our 43-bit primitive-based design achieves the highest frequency while maintaining moderate DSP usage and normalized area. It achieves 83% higher frequency than IMpress1, with 16% fewer LUTs and the same number of DSPs. Against IMpress2, it achieves 56% higher frequency while using 17% fewer DSPs and slightly fewer LUTs. It also achieves 62% higher frequency than the beam-search tiling heuristic in [2], while reducing DSP block usage by 45% and LUTs by 5%.

Generality and Extensibility. We also provide a preliminary evaluation using the DSP58 primitives on the AMD Versal (VCK190 dev board) to demonstrate the generality of the automated flow (direct comparison with the DSP48E2 results is not meaningful). These are 27×24-bit multipliers, and using our approach, a 49-bit unsigned base multiplier is constructed, requiring four DSP58 blocks combined with a 3×3 multiplier using LUTs. This optimized base is then scaled using overshoot tiling, as before. Table 2 shows

Table 2: Detailed comparison of different decomposition schemes for various multiplier sizes. Target-constrained tiling is used with the 77-bit base multiplier, while overshoot tiling is used with the 43-bit base.

Target Size	Actual Size	Base	#Karat. Levels	LUTs	FFs	DSPs	Freq. (MHz)	Lat. (Cyc)
160	162	27×18	0	3461	6112	61	830	35
	160	77	1	3770	7507	62	855	31
	172	43	2	5141	9202	45	862	30
256	264	27×18	0	9947	15710	161	760	64
	256	77	1	8473	17718	114	776	40
	258	43	2	10858	20171	90	842	35
384	384	77	2	21350	38330	279	583	56
	387	43	2	23208	43803	180	751	48
512	512	77	2	33122	61525	370	619	72
	516	43	3	38131	68870	270	629	60
1024	1024	77	3	123472	207765	1220	478	123
	1032	43	4	128252	221612	810	458	100
1024	1024	IP	4	105717	169858	1296	459	89
Related Work								
1024	HLS [25]		6	165618	N/A	729	298	24
	IMpress1 [25]		N/A	150400	N/A	810	250	23
	IMpress2 [25]		N/A	129338	N/A	945	295	22
	BeamSearch [2]		N/A	133975	N/A	1471	283	N/A
	OpenNTT [15]		N/A	12717	14217	80	320	689
Versal VCK190								
1024	1176	49	4	163712	274787	684	496	115

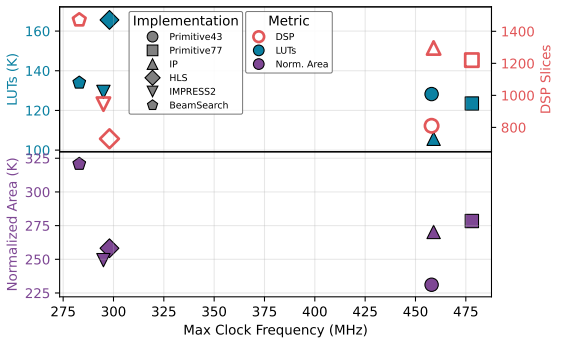


Figure 11: Area and performance comparison of different implementations showing 56% higher frequency while using 17% fewer DSPs and slightly fewer LUTs than the state of the art.

this DSP58-based implementation achieves high frequency and reduces DSP usage on the Versal, at the cost of a slight increase in LUTs.

4.2 Latency Mitigation

As shown, our primitive-based large multipliers achieve significantly higher frequencies than prior work, but they do suffer from higher latency in cycles. This can be mitigated by dividing combinational logic into larger segment sizes (e.g., 128 bits instead of 64 bits). Table 3 shows the impact of varying this on the latency and performance of a 1032-bit multiplier built using a 43-bit base. Increasing the segment size reduces the latency significantly, trading off slightly increased LUT usage and reduced maximum frequency. Using a 256-bit segment size reduces the latency from 100 to 58 cycles while still achieving higher frequency than other implementations. With the significant frequency improvement against previous work, this latency cost can be tolerable for the high throughput improvement.

Table 3: Impact of combinational logic segment size on latency and performance for 1032-bit multiplier 4-Karatsuba with 43-bit base

Segment Size	LUTs	FFs	Freq. (MHz)	Lat. (Cycles)
96	127225	225670	379	88
128	125480	219590	359	70
192	133741	216455	368	66
256	136056	213833	348	58

5 Related Work

Authors in [22] developed an automated tool for mapping arithmetic expressions to DSP blocks through DSP-aware pipelining. However, they relied on fixed template databases that constrain their generalizability and are not suitable for large integer multiplication. Authors in [25] presented IMpress, an E-graph framework for large integer multiplication using HLS. However, it suffers from low frequency. Extending general decomposition schemes to be architecture-aware has been explored in previous work. In [23], they proposed a general algorithm to tile base multipliers previously obtained in [7]. However, this is limited to Schoolbook decomposition, and is not suited to large numbers. Authors in [17] proposed a binary ILP formulation followed by beam search heuristic tiling [2] for integer multiplication. Both run at low frequency and are not area efficient in large sizes. Karatsuba-based tiling for DSP blocks has been explored in [16], but cannot scale to larger operands as it does not take advantage of hierarchical decomposition.

6 Conclusions and Future Work

Implementing large integer multiplication on FPGAs in a way that maximizes performance while also using resources efficiently is challenging. In this work, we developed an approach for building efficient square base multipliers suited to DSP block implementation, exploiting their architectural features. We automate the process of generating the RTL code for tiling these bases to build large-integer multipliers in an architecture-aware manner with fast pipelined combinational logic. The generated designs achieve high frequency, 56% higher than previous work, while using around 17% fewer DSP blocks. We can also save more resources by implementing the multiplier in a serial manner instead of fully unrolling the computation. We are releasing the tool for use by the wider research community.

In the future, we plan to leverage equality saturation to generalize this flow to arbitrary arithmetic expressions, exploiting architecture-aware rewriting in a similar manner. Finally, for very large multipliers, we can develop a higher level pipeline structure for better scaling across SLRs.

References

[1] Beaumont, Boudet, Rastello, and Robert. 2002. Partitioning a Square into Rectangles: NP-Completeness and Approximation Algorithms. *Algorithmica* (2002), 217–239. doi:10.1007/s00453-002-0962-9

[2] Andreas Böttcher, Keanu Kullmann, and Martin Kumm. 2020. Heuristics for the Design of Large Multipliers for FPGAs. In *IEEE Computer Arithmetic (ARITH)*. 17–24. doi:10.1109/ARITH48897.2020.00012

[3] Donald Donglong Chen, Gavin Xiaoxu Yao, Ray C.C. Cheung, Derek Pao, and Çetin Kaya Koç. 2016. Parameter Space for the Architecture of FFT-Based Montgomery Modular Multiplication. *IEEE Trans. Comput.* 65, 1 (2016), 147–160. doi:10.1109/TC.2015.2417553

[4] Gary C.T. Chow, Ken Eguro, Wayne Luk, and Philip Leong. 2010. A Karatsuba-Based Montgomery Multiplier. In *2010 International Conference on Field Programmable Logic and Applications*. 434–437. doi:10.1109/FPL.2010.89

[5] James W. Cooley and John W. Tukey. 1965. An Algorithm for the Machine Calculation of Complex Fourier Series. *Math. Comp.* (1965), 297–301. doi:10.2307/2003354

[6] S.N. Demidenko, D.Johnson, K.Gribbon, and Donald Bailey. 2002. *Implementing Signal Processing Algorithms in FPGAs: Digital Spectral Warping.*

[7] Florent Dinechin and Bogdan Pasca. 2009. Large Multipliers with Less DSP Blocks. In *Field Programmable Logic and Applications (FPL)*. doi:10.1109/FPL.2009.5272296

[8] Ando Emerencia. 2007. Multiplying huge integers using Fourier transforms.

[9] Niall Emmart and Charles C. Weems. 2011. High precision integer multiplication with a GPU using strassen's algorithm with multiple FFT sizes. *Parallel Processing Letters* (2011), 359–375. doi:10.1142/S0129626411000266

[10] Tassadaq Hussain, Saqib Amin, Usman Zabit, Faysal Kamran, Olivier D. Bernal, and Thierry Bosch. 2016. A High Performance Real-Time FPGA-Based Interferometry Sensor Architecture. In *International Conference on Frontiers of Information Technology (FIT)*. 130–135. doi:10.1109/FIT.2016.032

[11] Lenos Ioannou and Suhaib A Fahmy. 2022. Streaming Overlay Architecture for Lightweight LSTM Computation on FPGA SoCs. *ACM Transactions on Reconfigurable Technology and Systems* 16, 1 (2022), 8:1–8:26.

[12] MD. Mainul Islam, MD. Selim Hossain, MD. Shahjalal, MOH. Khalid Hasan, and Yeong Min Jang. 2020. Area-Time Efficient Hardware Implementation of Modular Multiplication for Elliptic Curve Cryptography. *IEEE Access* 8 (2020), 73898–73906. doi:10.1109/ACCESS.2020.2988379

[13] Jouni Isoaho, Jari Pasanen, Olli Vainio, and Hannu Tenhunen. 1993. DSP system integration and prototyping with FPGAs. *J. VLSI Signal Process. Syst.* (1993), 155–172.

[14] Anatolii Karatsuba and Yu Ofman. 1962. Multiplication of Multidigit Numbers on Automata. *Soviet Physics Doklady* 7 (1962), 595.

[15] Florian Krieger, Florian Hirner, Ahmet Can Mert, and Sujoy Sinha Roy. 2025. OpenNTT - An Automated Toolchain for Compiling High-Performance NTT Accelerators in FHE. In *International Conference on Computer-Aided Design (ICCAD '24)*. 1–9. doi:10.1145/3676536.3697123

[16] Martin Kumm, Oscar Gustafsson, Florent De Dinechin, Johannes Kappauf, and Peter Zipf. 2018. Karatsuba with Rectangular Multipliers for FPGAs. In *Computer Arithmetic (ARITH)*. 13–20. doi:10.1109/ARITH.2018.8464809

[17] Martin Kumm, Johannes Kappauf, Matei Istoan, and Peter Zipf. 2017. Resource Optimal Design of Large Multipliers for FPGAs. In *IEEE Computer Arithmetic (ARITH)*. 131–138. doi:10.1109/ARITH.2017.35

[18] Martin Langhammer and Bogdan Pasca. 2015. Floating-Point DSP Block Architecture for FPGAs. In *ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA)*. 117–125. doi:10.1145/2684746.2689071

[19] S.O. Memik, A.K. Katsaggelos, and M. Sarrafzadeh. 2003. Analysis and FPGA implementation of image restoration under resource constraints. *IEEE Trans. Comput.* (2003), 390–399. doi:10.1109/TC.2003.1183952

[20] Ahmet Can Mert, Erdinç Öztürk, and Erkay Savaş. 2020. Design and Implementation of Encryption/Decryption Architectures for BFV Homomorphic Encryption Scheme. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 28, 2 (2020), 353–362. doi:10.1109/TVLSI.2019.2943127

[21] Dhaval Patel, Bimal Patel, Jalpesh Vasa, and Mikin Patel. 2023. A Comparison of the Key Size and Security Level of the ECC and RSA Algorithms with a Focus on Cloud/Fog Computing. In *ICT with Intelligent Applications*. 43–53. doi:10.1007/978-981-99-3758-5_5

[22] Bajaj Ronak and Suhaib A. Fahmy. 2016. Mapping for Maximum Performance on FPGA DSP Blocks. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* (2016), 573–585. doi:10.1109/TCAD.2015.2474363

[23] Debapriya Basu Roy, Debdeep Mukhopadhyay, Masami Izumi, and Junko Takahashi. 2014. Tile Before Multiplication: An Efficient Strategy to Optimize DSP Multiplier for Accelerating Prime Field ECC for NIST Curves. In *Design Automation Conference (DAC)*. 1–6. doi:10.1145/2593069.2593234

[24] Russell Tessier and Wayne Burleson. 2001. Reconfigurable Computing for Digital Signal Processing: A Survey. *Journal of VLSI signal processing systems for signal, image and video technology* (2001), 7–27. doi:10.1023/A:1008155020711

[25] Ecenur Ustun, Ismail San, Jiaqi Yin, Cunxi Yu, and Zhiru Zhang. 2022. IMpress: Large Integer Multiplication Expression Rewriting for FPGA HLS. In *Field-Programmable Custom Computing Machines (FCCM)*. 1–10. doi:10.1109/FCCM53951.2022.9786123

[26] Dong Wang, Ke Xu, Jingning Guo, and Soheil Ghiasi. 2020. DSP-Efficient Hardware Acceleration of Convolutional Neural Network Inference on FPGAs. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* (2020), 4867–4880. doi:10.1109/TCAD.2020.2968023

[27] Xilinx, Inc. 2005. DSP: Designing for Optimal Results. High-Performance DSP Using Virtex-4 FPGAs (UG073).

[28] Xilinx, Inc. 2009. Virtex-5 FPGA User Guide (UG190).

[29] Xilinx, Inc. 2015. Multiplier v12.0 LogiCORE IP Product Guide (PG108).

[30] Xilinx, Inc. 2015. Virtex-6 Family Overview (DS150).

[31] Xilinx, Inc. 2019. Alveo U280 Data Center Accelerator Card Data Sheet(DS963).

[32] Xilinx, Inc. 2021. UltraScale Architecture DSP Slice User Guide (UG579).

[33] Xilinx, Inc. 2022. *Versal ACAP DSP Engine Architecture Manual (AM004).*

[34] Xilinx, Inc. 2024. UltraScale Architecture and Product Data Sheet: Overview (DS890). 44 pages.

[35] Chee Yap and Chen Li. 2000. QuickMul: Practical FFT-based Integer Multiplication.

[36] Andy G. Ye and David M. Lewis. 1999. Procedural texture mapping on FPGAs. In *Proceedings of ACM/SIGDA seventh international symposium on Field programmable gate arrays*. 112–120. doi:10.1145/296399.296438

[37] Tian Ye, Sanmukh R. Kuppannagari, Rajgopal Kannan, and Viktor K. Prasanna. 2021. Performance Modeling and FPGA Acceleration of Homomorphic Encrypted Convolution. In *2021 31st International Conference on Field-Programmable Logic and Applications (FPL)*. 115–121. doi:10.1109/FPL53798.2021.00027

[38] Pavel Zemcik. 2002. Hardware acceleration of graphics and imaging algorithms using FPGAs. In *Proceedings of Spring Conference on Computer Graphics*. 25–32. doi:10.1145/584458.584463